

Predicting LLM Response Quality Scores



*A Low-Cost, Interpretable Approach
to Real-Time LLM Quality Monitoring*

The Problem

Human Annotation

→ Costs \$M at scale, weeks to complete

LLM-as-Judge (GPT-4)

→ High API cost + model-specific bias

Neither scales to production monitoring.



Our Solution

Predict response quality scores from 17 interpretable text features.

No human evaluator. No secondary LLM. Real-time online inference.

Who uses this & Why ML?

End users: LLM product teams, model evaluation engineers, and AI governance teams needing fast, low-cost response monitoring.

Why ML: Rule-based scoring fails to generalize across diverse prompts and response formats — supervised learning approximates human preference signals at scale.

DATA & FINAL MODEL

Dataset: LMSYS Chatbot Arena

33K

Conversations

20+

LLMs

66K

Rows after
unpivoting

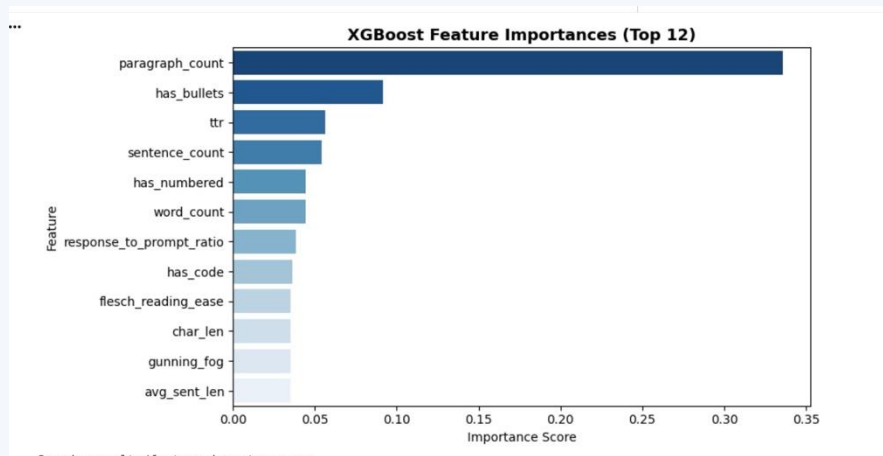
Target: `win_score` (1.0 win / 0.5 tie / 0.0 loss)

~48% ties — discrete, tie-heavy distribution

Model Comparison (held-out test set)

Model	RMSE ↓	Spearman ρ ↑
Ridge (baseline)	0.4150	0.1865
Random Forest ♦	0.4083	0.2416 ← best
XGBoost	0.4088	0.2337

Top Predictors (XGBoost Feature Importance)



Why Random Forest?

Best Spearman ρ = 0.2416

Tightest CV variance (± 0.0007)

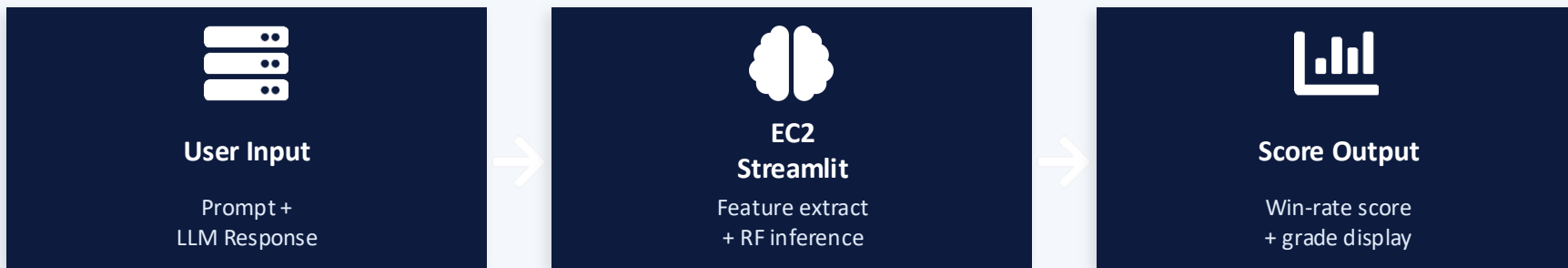
Native feature importance → interpretable

Fast inference — no GPU / API needed

① Training Path



② Inference Path



Service	Role	Status
Amazon S3	Model artifact storage	✓ Live
SageMaker Training	Managed training job (sklearn container)	✓ Used
EC2 (t2.micro)	Streamlit app — externally accessible :8501	✓ Live

LLM Response Quality Scorer

 About

Model: Random Forest Regressor
Target: Win-rate score (0 = loss, 0.5 = tie, 1.0 = win)
Features: 17 textual features
Spearman ρ : 0.2416 (best on test set)
Test RMSE: 0.4083
Dataset: LMSYS Chatbot Arena (33K conversations → 66K rows)

CMU Tepper MSBA
Machine Learning for Business
Group 15 — 2025

Feature categories:

-  Lexical (6 features)
-  Structural (6 features)
-  Readability (3 features)
-  Prompt-relative (2 features)

Deploy

LLM Response Quality Scorer

Predict the **human preference win-rate score** of an LLM response using interpretable textual features — no secondary LLM required.

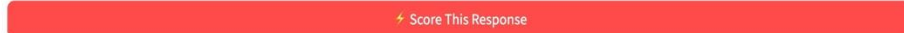
 Model loaded from local file

 Prompt (user question)

Enter the prompt / question that was given to the LLM...

 LLM Response

Paste the LLM response to evaluate here...



Demo Flow

1

Enter a prompt
& LLM response

2

Click "Score
This Response"

3

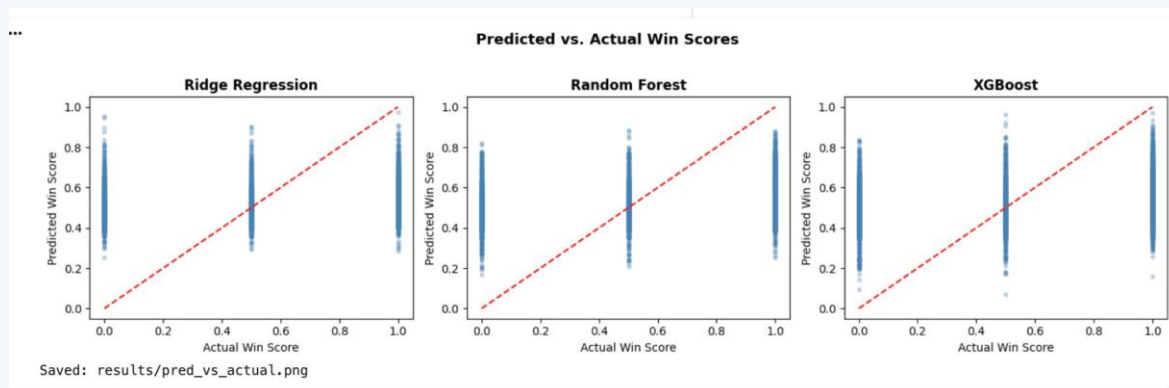
See win-rate score
& quality grade

4

Expand Feature
Breakdown panel

→ <http://54.210.43.192:8501>

MODEL RESULTS — DEEP DIVE



Predicted vs. Actual Win Scores

Random Forest: $\rho = 0.2416$

Best Spearman among all 3 models.
Correctly orders responses by quality
more reliably than Ridge or XGBoost.

RMSE Plateau ~ 0.41

Discrete target (0 / 0.5 / 1) creates
vertical scatter. All models partially
capture win direction.

Next: Semantic Features

BERTScore + perplexity expected to
meaningfully improve ρ beyond
current surface-level text signals.

LESSONS LEARNED & WHAT'S NEXT



What Worked



Framing pairwise votes as regression targets unlocked richer training signal than classification



17 lightweight features: fast inference with no GPU or API dependency



Random Forest's native feature importance made model behavior explainable



Streamlit + EC2 deployment was straightforward once Security Group was correctly configured



Harder Than Expected



Discrete target (0 / 0.5 / 1) limits RMSE improvement — Spearman ρ is the right metric



AWS Academy Security Group misconfig caused EC2 access issues (resolved)



~48% tie rate makes it hard to differentiate quality at the margins



What's Next



Add BERTScore + perplexity features to meaningfully improve Spearman ρ



Integrate RDS prediction logging for persistent score history & dashboard



Fine-tune a DeBERTa reward model as a stronger semantic baseline

THANK YOU

Questions?

66K

Training rows

17

Text features

0.2416

Spearman ρ

EC2

Live on AWS